॥ श्री गणेशाय नमः ॥

Asymptotic Analysis

Big Oh : $f(n) \le c \cdot g(n) \Rightarrow f(n) = O(n)$ Small oh : $f(n) < g(n) \Rightarrow f(n) = o(g(n))$

Big omega : $f(n) \ge c \cdot g(n) \Rightarrow f(n) = \Omega\, g(n)$ Small omega : $f(n) > g(n) \Rightarrow f(n) = \omega(g(n))$

Big Theta : $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n) \Rightarrow f(n) = \Theta(g)$

$f(n) = \omega(g(n)) \Rightarrow \lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$; $f(n) = o(g(n)) \Rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

* for direct comparisons, substitute really large values of n.

$$O(1) < O(\log\log n) < O(\log n) < O((\log n)^c) < O(n^c) < O(n) < O(n\log n) \equiv O(\log n!)$$
$$c > 1 \qquad n \in (0,1)$$

$$< O(n^2) < O(n^c) < O(c^n) < O(n!) \equiv O(n^n)$$

Master Theorem

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$
$$a \ge 1, \quad b > 1$$

compare $n^{\log_b a}$ and $f(n)$
- If one is polynomially greater → $T(n) = \Theta(\text{greater})$
- If no one is poly. greater → $T(n) = \Theta(\text{greater} * \log n)$

Arrays

Addressing $A[l_1 \dots u_1][l_2 \dots u_2]$

1-D: $A[i] = \text{base} + (i - l) * w$

2-D: row major → $A[i][j] = \text{base} + [(i-l_1)*c + (j-l_2)] * w$

col major → $A[i][j] = \text{base} + [(i-l_1) + (j-l_2)*r] * w$

$r = u_1 - l_1 + 1$
$c = u_2 - l_2 + 1$

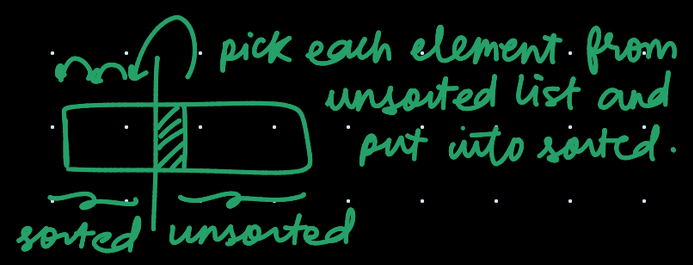
	Comparisons / Time	Space
Find minimum/maximum	$N-1$	$O(1)$
Find min & max both	$\lceil \frac{3N}{2} - 2 \rceil$	$O(1)$
Find 2nd min or 2nd max	$N + \lceil \log N \rceil - 2$	$O(N)$
Insert into sorted array	$O(N)$	$O(1)$

Linked lists

search → $O(n)$. Any operation that requires to go to an element takes $O(n)$ + operation time.

Sorting

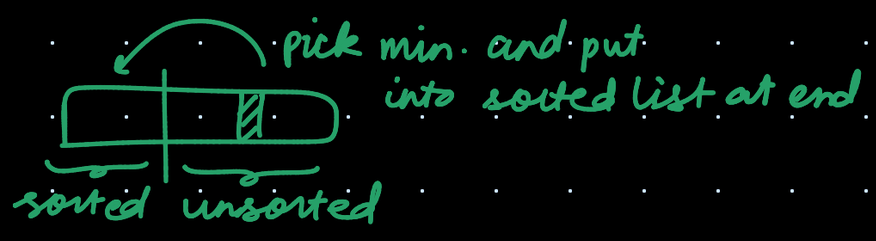
Insertion sort



Best : O(n)
worst / avg : $O(n^2)$
Extra space : O(1)

in-place ✓
stable ✓

Selection sort



Best : $O(n^2)$
worst / avg : $O(n^2)$
Extra space : O(1)

in-place ✓
stable ✗

Bubble Sort



Best : $O(n^2)$
worst / avg : $O(n^2)$
Extra space : O(1)

in-place ✓
stable ✓

Quick sort

Best : O(nlogn) O(n) → 3-way with equal keys
worst : $O(n^2)$ avg : O(nlogn) In-place ?
Extra space : O(1) Stable ✗

$T(n) = 2 \cdot T(\frac{n}{2}) + O(n)$: best / avg

$T(n) = T(n-1) + cn$: worst

Paitition algo : $T = O(n)$

Merge sort

Best : O(nlogn)
worst / avg : O(nlogn)
Extra space : O(n)
O(1) with LL

In-place : with array ✗ with LL ✓

stable : ✓

$T(n) = 2 \cdot T(\frac{n}{2}) + O(n)$

Stacks (LIFO) 

push = O(1), pop = O(1).

Valid stack permutations : $\frac{^{2n}C_n}{(n+1)}$ out of n! total.

Queue (FIFO)

enqueue = O(1), dequeue = O(1).

Infix to postfix

* A higher priority operator should sit on top of a lower priority one.
to push lower, pop higher first.

```
For char c in expression:
    if c is an operand
        output c
    if right parenthesis:
        pop and output till left parenthesis
    if operator or left parenthesis:
        pop until lower parenthesis or lower priority encountered.
        push c
```

Evaluation:

Postfix → scan left to right Prefix → scan right to left.
Push operands. Pop as reqd when operator.

Queue using 2 stacks (S_1, S_2)

To enqueue:
```
S1.push()
```

To dequeue:
```
S2.dequeue()
If S2 is empty:
    S1 ⟶ S2  empty S1 into S2
    S2.dequeue()
```

Trees

Complete BT
→ All leaves at same level.
→ All non-leaves have 2 children.

Almost complete BT
→ All leaves at last level or last-1th level.
→ Filled L→R

$\#$ of nodes $= 2^{h+1} - 1$

Height $= \lfloor \log_2 n \rfloor$

$\#$ of binary trees possible using n nodes → (Catalan $\#$)

$\frac{^{2n}C_n}{(n+1)}$ unlabeled $\frac{^{2n}C_n}{(n+1)} \cdot n!$ labeled

Tree traversals → In-order, Pre-order, Post-order

Time: $O(n)$
Space: $O(n)$ skewed
Avg case: $\Theta(\log n)$

In + Pre/Post → reqd. to reconstruct

BST

- In-order is sorted in increasing order.
- Search: $O(n)$ worst, $O(\lg n)$ avg/best-
- Constructing a BST: $O(n \lg n)$ best-, $O(n^2)$ worst-
- Insertion: $O(1)$ best, $O(\lg n)$ avg, $O(n)$ worst
- Deletion: replace with in-order successor/predecessor. $O(n)$ worst
- To find min: $O(n)$ worst Keep going left $O(\text{height})$

AVL trees

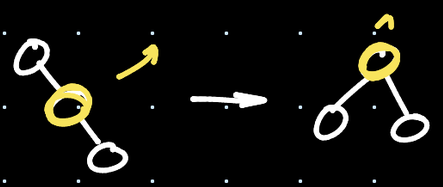
BF = height of left subtree − height of right subtree

BF = {−1, 0, 1} allowed for AVL.

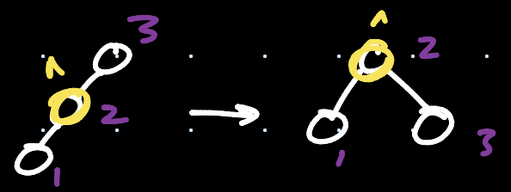
Height $= O(\log n)$
Search $= O(\log n)$

Insertion: $O(\log n)$ insertion + $O(\log n)$ find imbalance + $O(1)$ rotate

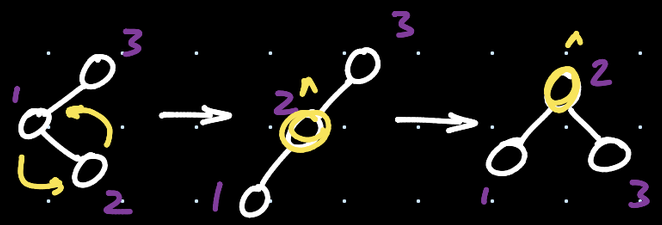
RR imbalance



LL imbalance



LR imbalance



RL imbalance



Heaps

Almost complete BT that follows heap property

- max heap property: node ≥ all nodes in sub-tree

Array representation:
left_child(i) = 2i
right_child(i) = 2i-1
parent(i) = $\lfloor \frac{i}{2} \rfloor$

- Non-leaves: 1 to $\lfloor \frac{n}{2} \rfloor$ leaves: $\lfloor \frac{n}{2}+1 \rfloor$ to n.

- max heapify:
  compare a node with its children. Swap with largest. Repeat down the tree.

  Time = O(logn) space = O(1)

- Build max heap: Apply max_heapify for nodes $\lfloor \frac{n}{2} \rfloor$ to 1.

  Time = O(n) space = O(logn).

- To extract max/root:
  Replace root with last node, delete it, and apply max heapify.

  Time = O(logn) space = O(logn)

- Increase_key:
  change value. Keep swapping with parent if needed. (recursively)

  Time = O(logn) space = O(1)

  Decrease_key:
  Decrease and call heapify on it.

- Insert-
  Add -∞ to end. Increase_key (-∞ → reqd. value)

  Time = O(logn) space = O(1).

- Delete: Find. Replace with last node and call heapify.
  O(n) (linear search) O(logn)

  Time = O(n)

Heap sort

Build max heap. Now top element is max. Swap with last element. Decrease heap size. Apply max_heapify on the root. Repeat.

max_heapify takes O(height) = O(logn). Each time height reduces by 1.

Time = O(nlogn) → distinct keys } Best worst/avg = O(nlogn)
O(n) → equal keys:

Auxiliary space: O(1)

In-place ✓
stable ×

Hashing

Hash functions:
→ Division modulo: $h(key) = key \bmod n$ → Pref. a prime far from any power of 2
→ Mid-square method: square the key and choose the middle digits.
→ Digit extraction: choose digits at particular places. (Not good)
→ Fold boundary: Add up boundaries. (Not good)
→ Fold shifting: add up folds.

Among these, mid-square and fold-shifting are comparatively better.

Collision Resolution:

→ Chaining (linked list / outside / external)
keys per slot = $\frac{n \text{ keys}}{m \text{ slots}} = \frac{n}{m} = \alpha$ $\alpha \in [0, \infty)$ In open-addressing, $\alpha = 1$

→ Open addressing (internal) →
- linear probing $LP(key, i) = [h(k) + i] \bmod m$
- quadratic probing $QP(key, i) = [h(k) + i^2] \bmod m$
- double hashing $h_1(k) = k \bmod m$ $h_2(k) = (1 + k) \bmod (m-2)$ $DH(k) = [h_1(k) + i \cdot h_2(k)] \bmod m$

Quadratic probing may never reach an empty slot.

Greedy algorithms:

Fractional Knapsack: $O(n \log n)$
Huffman coding: $O(n \log n)$ n = # of unique char
Job sequencing with deadlines: $O(n \log n)$ to sort jobs acc to profit.
$O(n^2)$ go to deadline and scan backwards for vacant slot